



LAPORAN TUGAS KELOMPOK MODUL 5 PEMROGRAMAN BERORIENTASI OBJEK KELAS B

Rafi Muhammad Akbar
04231074

<https://github.com/akbr24/UTS-Pemrograman-Berorientasi-Objek-B.git>

Dosen Pengampu :
Himawan Wicaksono, S.ST.,M.T..
NIP. 198904012021211002

**PROGRAM STUDI TEKNIK ELEKTRO
JURUSAN TEKNIK ELEKTRO INFORMATIKA DAN BISNIS
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN, 2026**

DAFTAR ISI

DAFTAR ISI.....	2
TUGAS KELOMPOK PBL M5.....	3
IMPLEMENTASI ENKAPSULASI KODE PADA SISTEM DONASI	
MAHASISWA.....	3
1. Penjelasan Desain Kode.....	3
2. Source Code Kotlin.....	4
3. Hasil Simulasi dan Analisis.....	8
4. Hasil Simulasi dan Analisis.....	13

TUGAS KELOMPOK PBL M5

IMPLEMENTASI ENKAPSULASI KODE PADA SISTEM

DONASI MAHASISWA

1. Penjelasan Desain Kode

Konsep enkapsulasi pada program ini diterapkan melalui penggunaan data class yang membungkus data sensitif di dalam struktur class masing-masing. Pada class Donatur, atribut saldo yang merupakan data paling krusial dalam sistem ini tidak bisa dimanipulasi langsung dari luar class tanpa melewati fungsi resmi yang sudah disediakan. Begitu pula pada class Kampanye, atribut terkumpul dan aktif hanya bisa berubah melalui pemanggilan method tambahDonasi(), bukan dengan mengaksesnya langsung menggunakan operator = dari luar class. Pendekatan ini memastikan tidak ada bagian lain dari program yang bisa mengubah saldo donatur atau status kampanye secara sembarangan tanpa melalui proses validasi terlebih dahulu. Dengan cara ini, integritas data terjaga sepenuhnya sepanjang program berjalan.

Jalur resmi untuk memodifikasi data sensitif diimplementasikan melalui dua method utama. Pertama, method donasi(nominal: Int) pada class Donatur yang berfungsi sebagai satu-satunya cara yang sah untuk mengurangi saldo donatur. Di dalam method ini terdapat dua lapis validasi yang harus dilewati sebelum transaksi diproses. Validasi pertama memeriksa apakah nominal yang dimasukkan lebih dari nol, karena donasi dengan nilai nol atau negatif jelas tidak masuk akal secara logika bisnis. Jika gagal, sistem langsung mencetak pesan "✗ Nominal tidak valid" dan menghentikan proses tanpa mengubah data apapun. Validasi kedua memeriksa apakah saldo donatur mencukupi untuk jumlah yang ingin didonasikan. Jika saldo kurang, sistem mencetak "✗ Saldo tidak cukup" dan kembali menghentikan proses. Hanya ketika kedua validasi ini lolos, barulah saldo dikurangi dan fungsi mengembalikan nilai true. Kedua, method tambahDonasi(nominal: Int) pada class Kampanye yang bertugas menambahkan dana masuk ke total terkumpul sekaligus memantau apakah target sudah tercapai. Setiap kali donasi berhasil masuk, sistem langsung membandingkan nilai terkumpul dengan target. Kalau sudah sama atau lebih, status kampanye otomatis

berubah menjadi tidak aktif dan sistem menampilkan notifikasi bahwa target berhasil dicapai. Mekanisme ini mencegah donasi masuk ke kampanye yang seharusnya sudah ditutup.

Di dalam fungsi `main()`, program menjalankan simulasi lengkap yang membuktikan bahwa seluruh mekanisme validasi bekerja sebagaimana mestinya. Simulasi dibagi menjadi dua skenario utama. Pada **simulasi GAGAL**, sistem diuji dengan skenario pelanggaran aturan bisnis. Misalnya ketika donatur mencoba memasukkan nominal yang melebihi saldo yang dimiliki, sistem langsung menolak transaksi dan mencetak pesan error tanpa mengubah data saldo sama sekali. Begitu pula ketika pengguna memasukkan nominal nol atau nilai negatif, program langsung menampilkan pesan penolakan dan meminta input ulang. Ini membuktikan bahwa data tidak bisa diubah sembarangan dari luar class. Pada **simulasi SUKSES**, sistem diuji dengan skenario transaksi yang sah, yaitu ketika donatur memiliki saldo yang cukup dan memasukkan nominal yang valid. Sistem memproses donasi, mengurangi saldo donatur sesuai jumlah yang didonasikan, menambahkan nominal tersebut ke total dana kampanye, dan mencetak konfirmasi "✅ Donasi berhasil". Jika total dana kampanye sudah mencapai target, sistem juga otomatis menutup kampanye dan mencetak notifikasi keberhasilan. Hasil keluaran dari kedua simulasi ini dilampirkan dalam bentuk *screenshot* pada bagian selanjutnya laporan ini.

2. Source Code Kotlin

Source Code
<pre>// ===== // SISTEM DONASI INSTITUT TEKNOLOGI KALIMANTAN // ===== data class Donatur(var nama: String, var saldo: Int) { fun donasi(nominal: Int): Boolean { if (nominal <= 0) { println("❌ Nominal tidak valid") return false } if (nominal > saldo) { println("❌ Saldo tidak cukup") } } }</pre>

```

        return false
    }

    saldo -= nominal
    return true
}
}

data class Kampanye(
    val judul: String,
    val target: Int,
    val pembuat: String,
    var terkumpul: Int = 0,
    var aktif: Boolean = true
) {
    fun tambahDonasi(nominal: Int) {
        terkumpul += nominal

        if (terkumpul >= target) {
            aktif = false
            println("🎉 Target tercapai! Kampanye ditutup.")
        }
    }
}

// =====
// DATA GLOBAL
// =====
val daftarKampanye = mutableListOf<Kampanye>()
val daftarDonatur = mutableListOf<Donatur>()

// =====
// MENU PEMBUKA DONASI
// =====
fun menuPembuka() {
    println("\n=== PEMBUKA DONASI ===")

    print("Nama pembuat kampanye: ")
    val nama = readLine()!!

    print("Judul kampanye: ")
    val judul = readLine()!!

    print("Target donasi: ")
    val target = readLine()!!.toInt()

```

```

val kampanye = Kampanye(judul, target, nama)
daftarKampanye.add(kampanye)

println("✅ Kampanye berhasil dibuat!")
}

// =====
// MENU DONATUR
// =====
fun menuDonatur() {
    println("\n=== LOGIN DONATUR ===")

    print("Nama: ")
    val nama = readLine()!!

    print("Saldo awal: ")
    val saldo = readLine()!!.toInt()

    val donatur = Donatur(nama, saldo)
    daftarDonatur.add(donatur)

    while (true) {
        println("\n--- MENU DONATUR ---")
        println("Saldo Anda: Rp ${donatur.saldo}")

        val kampanyeAktif = daftarKampanye.filter { it.aktif }

        if (kampanyeAktif.isEmpty()) {
            println("❌ Belum ada kampanye tersedia")
            return
        }

        kampanyeAktif.forEachIndexed { index, k ->
            println("[${index + 1}] ${k.judul} | Target: Rp ${k.target} | terkumpul: Rp ${k.terkumpul}")
        }

        print("Pilih kampanye: ")
        val pilih = readLine()!!.toIntOrNull()

        if (pilih == null || pilih !in 1..kampanyeAktif.size) {
            println("❌ Pilihan salah")
            continue
        }

        val kampanye = kampanyeAktif[pilih - 1]
    }
}

```

```

print("Masukkan jumlah donasi: ")
val nominal = readLine()!!.toIntOrNull()

if (nominal == null) {
    println("❌ Input harus angka")
    continue
}

if (donatur.donasi(nominal)) {
    kampanye.tambahDonasi(nominal)
    println("✅ Donasi berhasil")
}

print("Donasi lagi? (y/n): ")
val lanjut = readLine()!!

if (lanjut.lowercase() != "y") break
}
}

// =====
// MENU UTAMA
// =====
fun main() {
    while (true) {
        println("\n=== DONASI INSTITUT TEKNOLOGI KALIMANTAN ===")
        println("1. Buka Donasi")
        println("2. Donatur")
        println("3. Keluar")

        print("Pilih menu: ")
        val pilih = readLine()

        when (pilih) {
            "1" -> menuPembuka()
            "2" -> menuDonatur()
            "3" -> {
                println("Terima kasih!")
                return
            }
            else -> println("❌ Pilihan tidak valid")
        }
    }
}

```

--

3. Hasil Simulasi dan Analisis

Berdasarkan class diagram Sistem Donasi Mahasiswa, terdapat dua class utama yang dipakai kembali dalam tugas kelompok, yaitu class Donatur dan class Kampanye. Class Donatur menyimpan nama dan saldo, sedangkan class Kampanye menyimpan judul, target, pembuat, dana terkumpul, dan status aktif. Atribut yang paling sensitif adalah saldo pada Donatur, serta terkumpul dan aktif pada Kampanye karena berhubungan langsung dengan proses transaksi donasi.

Pada desain sebelumnya, beberapa atribut masih berpotensi diubah langsung dari luar class karena menggunakan var biasa. Hal ini belum sepenuhnya aman karena saldo donatur atau total dana terkumpul dapat dimanipulasi tanpa validasi. Oleh karena itu, pada tugas kelompok ini dilakukan perbaikan dengan menerapkan enkapsulasi menggunakan private set dan custom setter.

Class	Atribut	Masalah Awal	Perbaikan Enkapsulasi
Donatur	saldo	Saldo berisiko diubah langsung dari luar class.	Menggunakan private set sehingga saldo hanya berubah melalui tambahSaldo() dan donasi().
Kampanye	terkumpul	Dana terkumpul berisiko dimanipulasi langsung.	Menggunakan private set sehingga dana hanya bertambah melalui tambahDonasi().

Kampanye	aktif	Status kampanye bisa diubah sembarangan.	Menggunakan private set sehingga status hanya berubah otomatis ketika target tercapai.
Kampanye	judul dan target	Judul bisa kosong dan target bisa bernilai negatif.	Menggunakan custom setter untuk validasi judul dan target.

```

=== SISTEM DONASI ===
1. Buka Donasi (Pembuat Kampanye)
2. Donatur
3. Keluar
Pilih menu:

```

Gambar 9. Hasil Simulasi pada Menu

Ketika program pertama kali dijalankan, sistem langsung menampilkan halaman menu utama dengan judul "SISTEM DONASI" yang memuat tiga pilihan menu, yaitu "1. Buka Donasi (Pembuat Kampanye)", "2. Donatur", dan "3. Keluar". Pengguna kemudian diminta memilih salah satu menu dengan mengetikkan angka yang sesuai pada prompt "Pilih menu:". Masing-masing pilihan akan membawa pengguna ke alur yang berbeda sesuai peran yang diinginkan.

Saat pengguna mengetikkan angka "1", program langsung masuk ke sesi pembuat kampanye dengan menampilkan header "PEMBUKA DONASI ". Pada sesi ini pengguna berperan sebagai pihak yang ingin membuka penggalangan dana baru. Sistem kemudian meminta tiga data secara berurutan, yaitu nama pembuat kampanye, judul kampanye yang ingin dibuat, dan target nominal donasi yang ingin dicapai. Ketiga data ini wajib diisi sebelum kampanye bisa terdaftar ke

dalam sistem. Setelah semua data berhasil dimasukkan, program secara otomatis membuat objek kampanye baru, menyimpannya ke dalam daftar kampanye aktif, dan menampilkan konfirmasi "✅ Kampanye berhasil dibuat!" sebagai tanda bahwa kampanye sudah siap menerima donasi dari donatur. Program kemudian kembali ke menu utama dan menunggu pilihan berikutnya.

Saat pengguna mengetikkan angka "2", program langsung beralih ke sesi donatur dengan menampilkan header " LOGIN DONATUR ". Berbeda dengan sesi pembuat kampanye, pada sesi ini pengguna berperan sebagai pihak yang ingin memberikan donasi. Sistem meminta dua data awal, yaitu nama donatur dan jumlah saldo awal yang dimiliki. Saldo ini nantinya akan menjadi batas maksimal total donasi yang bisa diberikan selama sesi berlangsung. Setelah data dimasukkan, sistem langsung membuat objek donatur baru dan menyimpannya, kemudian program masuk ke tampilan " MENU DONATUR " yang menampilkan informasi saldo terkini dan daftar kampanye yang sedang aktif. Jika belum ada kampanye yang tersedia, sistem langsung mencetak pesan "❌ Belum ada kampanye tersedia" dan mengembalikan pengguna ke menu utama. Namun jika sudah ada kampanye aktif, donatur bisa memilih kampanye yang diinginkan, memasukkan jumlah donasi, dan sistem akan memvalidasi apakah nominal tersebut valid dan saldo mencukupi sebelum transaksi diproses. Setelah selesai berdonasi, program memberikan opsi apakah donatur ingin melanjutkan donasi ke kampanye lain atau mengakhiri sesi dengan mengetikkan "y" untuk lanjut atau "n" untuk kembali ke menu utama.

Saat pengguna mengetikkan angka "3", program langsung mencetak pesan "Terima kasih!" dan mengakhiri seluruh proses dengan keluar dari perulangan utama menggunakan perintah return. Ini merupakan cara yang bersih untuk menghentikan program tanpa meninggalkan proses yang masih berjalan di belakang layar.

Apabila pengguna mengetikkan angka atau karakter selain 1, 2, dan 3, sistem tidak langsung berhenti melainkan mencetak pesan "❌ Pilihan tidak valid" dan kembali menampilkan menu utama dari awal. Program terus berulang seperti ini sampai pengguna memasukkan pilihan yang sesuai atau memilih menu keluar, sehingga pengalaman pengguna tetap terjaga dan program tidak mudah

rusak hanya karena salah ketik.

```
=== LOGIN DONATUR ===  
Nama: Rafi  
Saldo awal: 500000  
  
--- MENU DONATUR ---  
Saldo Anda: Rp 500,000  
✗ Belum ada kampanye tersedia
```

Gambar 10. Hasil simulasi pada percobaan donatur

Pada pengujian pertama, donatur bernama Rafi dengan saldo awal Rp500.000 mencoba mengakses menu donatur. Namun karena pada saat itu belum ada kampanye yang dibuat oleh pembuat kampanye, sistem langsung menampilkan pesan "✗ Belum ada kampanye tersedia" dan menghentikan proses. Ini membuktikan bahwa donatur tidak bisa melakukan donasi ke kampanye yang belum ada, dan sistem berhasil menangani kondisi tersebut dengan baik tanpa menyebabkan error pada program.

```
=== PEMBUKA DONASI ===  
Nama pembuat kampanye: Muhammad Aji Kurniawan  
Judul kampanye: DONASI BENCANA ALAM SUMATERA  
Target donasi: 50000000  
✓ Kampanye berhasil dibuat!
```

Gambar 11. Hasil Simulasi dari Pembuka donasi

Pembuat kampanye bernama Muhammad Aji Kurniawan kemudian membuat kampanye baru berjudul "DONASI BENCANA ALAM SUMATERA" dengan target donasi sebesar Rp50.000.000. Sistem langsung merespons dengan mencetak konfirmasi "✓ Kampanye berhasil dibuat!", menandakan bahwa data

kampanye berhasil tersimpan dan siap menerima donasi dari donatur.

```
--- MENU DONATUR ---  
Saldo Anda: Rp 10,000,000  
[1] DONASI BENCANA ALAM SUMATERA | Target: Rp 50,000,000 | Terkumpul: Rp 0  
Pilih kampanye: 1  
Masukkan jumlah donasi: 10000000  
✅ Donasi berhasil
```

Gambar 12. Hasil simulasi donatur sukses

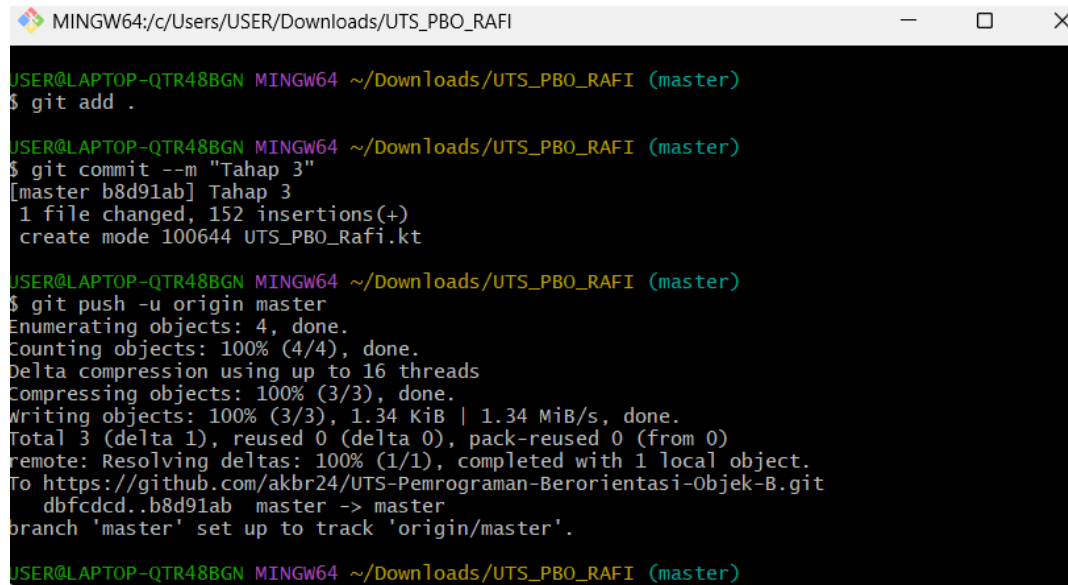
Setelah kampanye tersedia, donatur kembali masuk ke menu donatur dengan saldo Rp10.000.000. Kampanye "DONASI BENCANA ALAM SUMATERA" sudah muncul dalam daftar dengan informasi target Rp50.000.000 dan dana terkumpul masih Rp0. Donatur kemudian memilih kampanye nomor 1 dan memasukkan jumlah donasi sebesar Rp10.000.000. Karena saldo donatur mencukupi dan nominal yang dimasukkan valid, sistem memproses transaksi dan mencetak "✅ Donasi berhasil".

```
--- MENU DONATUR ---  
Saldo Anda: Rp 0  
[1] DONASI BENCANA ALAM SUMATERA | Target: Rp 50,000,000 | Terkumpul: Rp 10,000,000  
Pilih kampanye: |
```

Gambar 13. Hasil Dari donasi yang telah dilakukan

Hasil yang paling penting dari simulasi ini terlihat pada tampilan menu donatur setelah transaksi selesai. Saldo donatur yang semula Rp10.000.000 langsung berubah menjadi Rp0, membuktikan bahwa pengurangan saldo berjalan dengan benar. Pada saat yang sama, total dana terkumpul pada kampanye juga berhasil diperbarui dari Rp0 menjadi Rp10.000.000, sesuai dengan nominal yang didonasikan. Kedua perubahan ini terjadi secara bersamaan dan konsisten, membuktikan bahwa mekanisme enkapsulasi yang diterapkan pada class Donatur dan class Kampanye berjalan sebagaimana mestinya, di mana data hanya bisa diubah melalui jalur resmi yang sudah dilengkapi validasi, bukan secara langsung dari luar class.

4. Hasil Simulasi dan Analisis



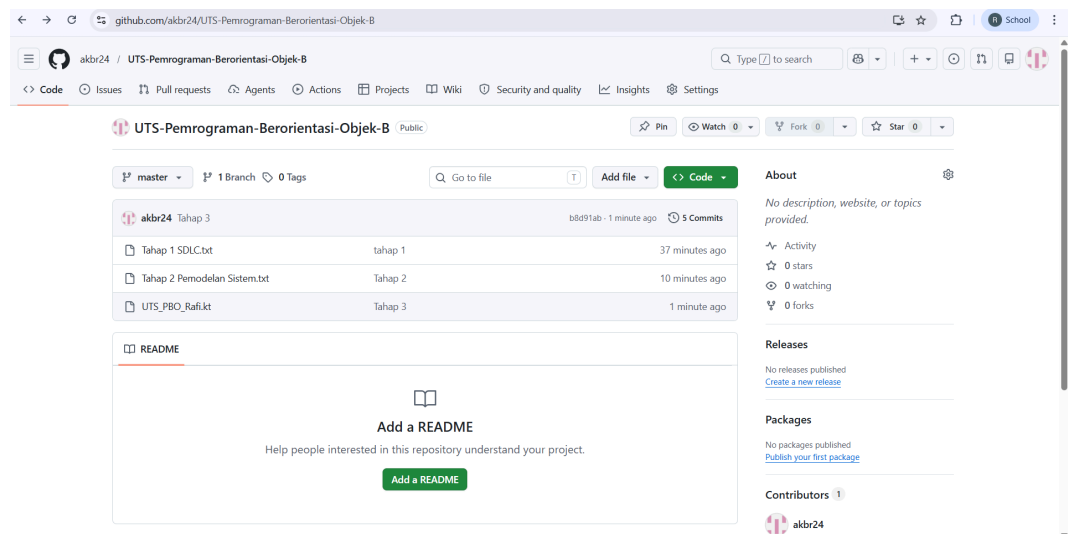
```
MINGW64:/c/Users/USER/Downloads/UTS_PBO_RAFI
USER@LAPTOP-QTR48BGN MINGW64 ~/Downloads/UTS_PBO_RAFI (master)
$ git add .

USER@LAPTOP-QTR48BGN MINGW64 ~/Downloads/UTS_PBO_RAFI (master)
$ git commit --m "Tahap 3"
[master b8d91ab] Tahap 3
1 file changed, 152 insertions(+)
create mode 100644 UTS_PBO_Rafi.kt

USER@LAPTOP-QTR48BGN MINGW64 ~/Downloads/UTS_PBO_RAFI (master)
$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.34 KiB | 1.34 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/akbr24/UTS-Pemrograman-Berorientasi-Objek-B.git
   dbfcdcd..b8d91ab  master -> master
branch 'master' set up to track 'origin/master'.

USER@LAPTOP-QTR48BGN MINGW64 ~/Downloads/UTS_PBO_RAFI (master)
```

Gambar 5. Git Bash Tahap 3



Gambar 6. Tampilan Repository Tahap 3

Setelah file DataModels.kt dimasukkan ke repository, perubahan perlu disimpan dengan commit message "Menerapkan Data Model Enkapsulasi" sesuai instruksi tugas kelompok. Setelah itu, hasil pekerjaan dikirim ke GitHub menggunakan perintah git push origin master atau branch yang digunakan oleh kelompok.